

Universidad San Carlos de Guatemala

Nombre: Jonathan Eduardo Fuentes García

Carnet: 202408977

Curso: Lenguajes Formales y de Programación

Auxiliar: Carlos Javier Cox Bautista

Tema:



MANUAL TECNICO

Fecha: 4/05/2026

Primer semestre

Introducción

Este documento explica cómo está construido por dentro el proyecto **TaskScript**. TaskScript es un programa de escritorio que lee archivos de texto con extensión .task, los analiza y genera reportes visuales.

El programa revisa que los archivos estén escritos correctamente según las reglas del mini-lenguaje TaskScript. Para ello tiene dos etapas: primero revisa palabra por palabra (análisis léxico) y luego revisa la estructura general (análisis sintáctico). También guarda una lista de los errores encontrados y, si todo está bien, crea tres reportes en formato HTML más un diagrama del árbol de cómo se organizó el archivo.

Aquí se describen las piezas que forman el sistema, los planos (diagramas) que muestran cómo se conectan esas piezas y las reglas gramaticales que sigue el lenguaje TaskScript.

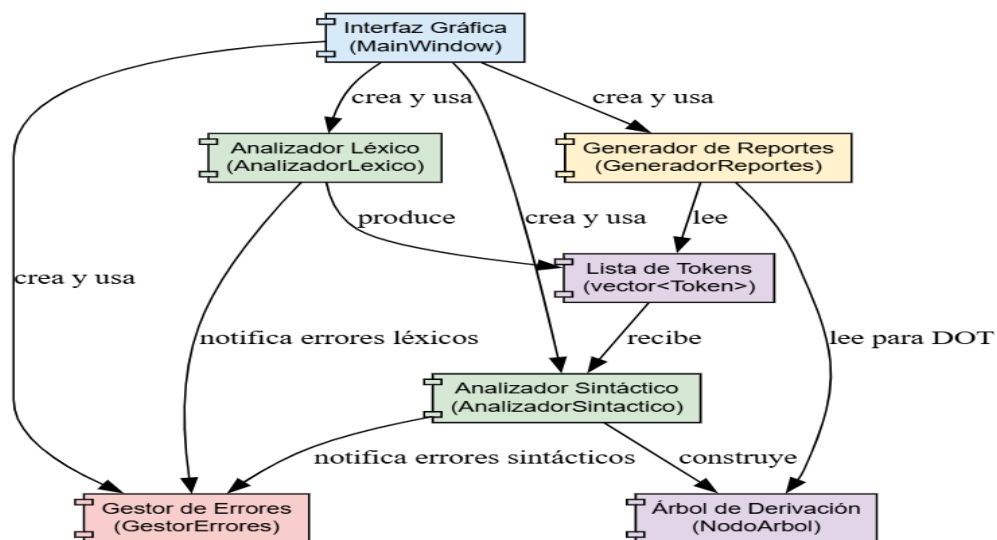
1. Arquitectura del sistema – ¿Cómo está organizado por dentro?

El sistema está dividido en dos grandes bloques:

- **El cerebro (backend):** Es la parte que hace los cálculos y los análisis. Está hecha en C++ estándar y contiene:
 - El analizador léxico (separa palabras y símbolos).
 - El analizador sintáctico (revisa el orden de las palabras).
 - El gestor de errores (anota todo lo que está mal).
 - El generador de reportes (fabrica los archivos HTML y el diagrama).
- **La cara visible (frontend):** Es la ventana con botones y tablas que ves en la pantalla. Está hecha con Qt.

La ventana toma el texto que escribiste o cargaste, se lo pasa al cerebro para que lo analice y luego muestra los resultados en las tablas.

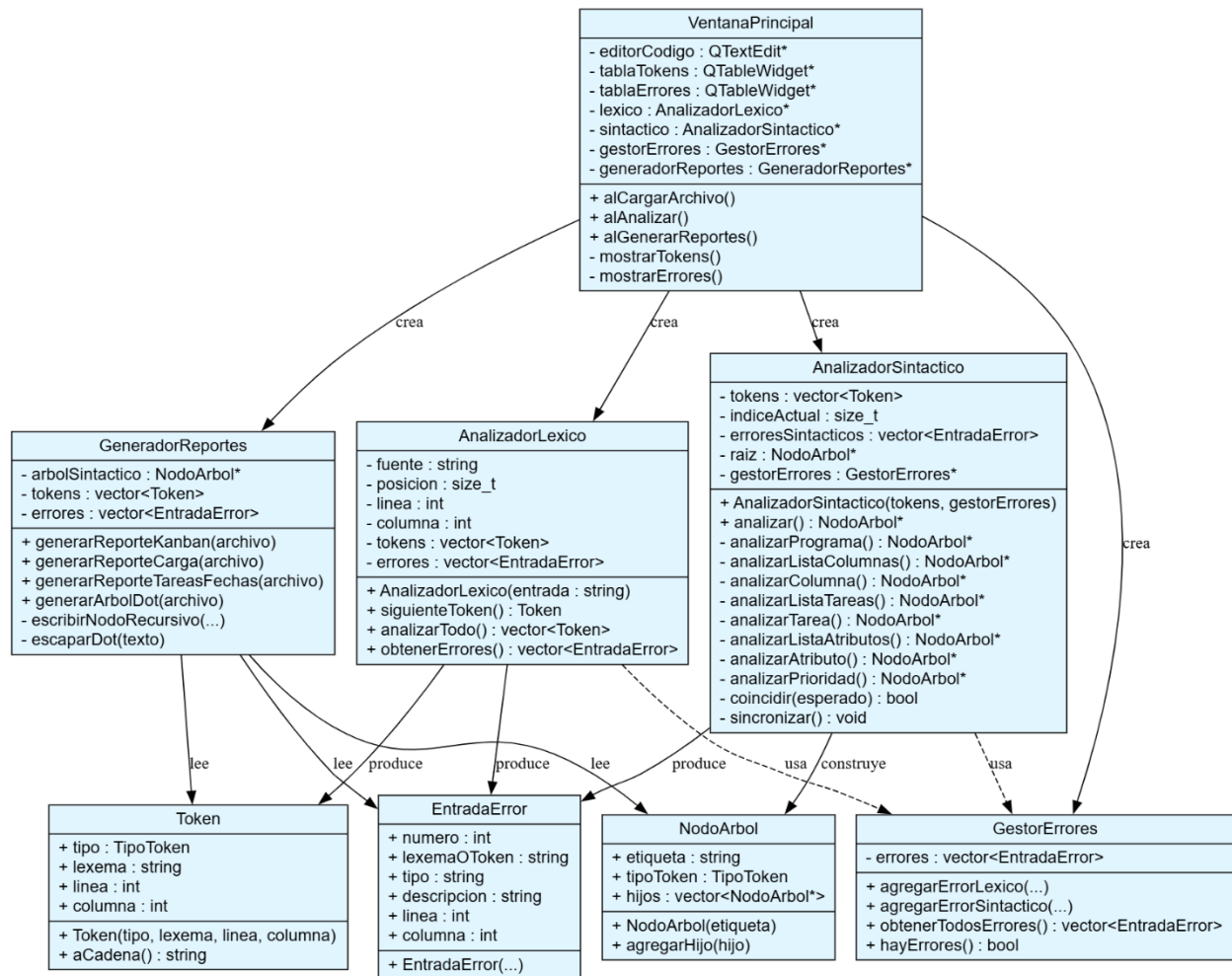
Estas dos partes se comunican de manera muy simple: el léxico le entrega una lista de "fichas" (tokens) al sintáctico, y ambos le avisan al gestor de errores cuando algo no cuadra.



2. Planos de las clases – Diagrama de clases

Las clases son como las piezas de un juego de construcción. Cada una hace una tarea específica:

Clase	¿Qué hace?
Token	Guarda una palabra o símbolo encontrado, su tipo, y en qué línea y columna apareció.
AnalizadorLexico	Lee el archivo carácter por carácter y va armando los tokens (las fichas).
AnalizadorSintactico	Revisa que los tokens estén en el orden correcto, como un inspector de gramática.
GestorErrores	Lleva una lista de todos los errores que se han encontrado.
GeneradorReportes	Crea los reportes HTML y el archivo DOT para dibujar el árbol.
MainWindow	Es la ventana principal de la aplicación, con botones y tablas.
NodoArbol	Representa una ramita del árbol de derivación.



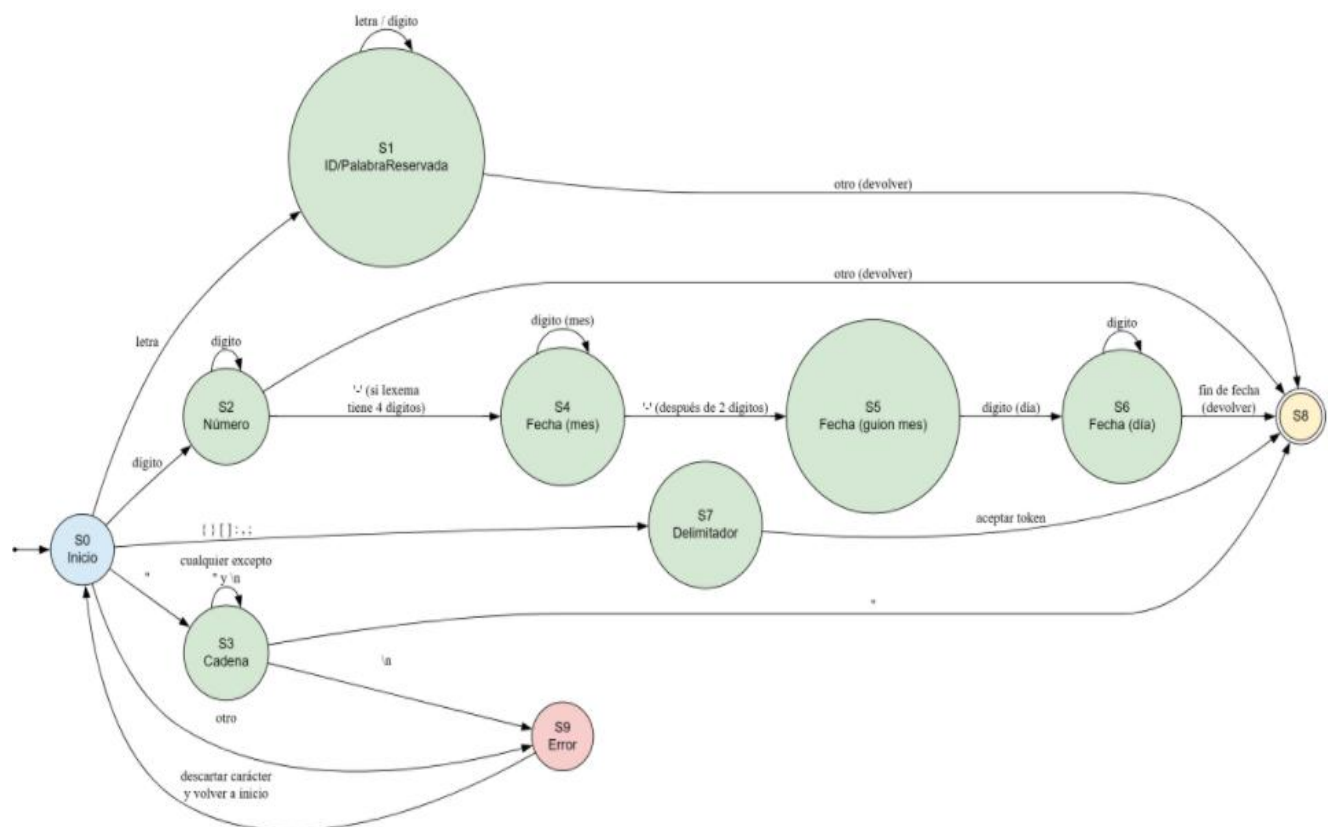
3. El autómata finito determinista (AFD)

El AFD es como un robot que solo sabe leer letra por letra y decidir qué tipo de palabra está formando. Tiene varios estados:

- **S0 (Inicio):** El robot acaba de despertar y mira el primer carácter.
- **S1 (Identificador):** Está leyendo una palabra que puede ser TABLERO, COLUMNA, tarea, etc.
- **S2 (Número):** Está leyendo dígitos. Si encuentra un guion, puede estar frente a una fecha.

- **S3 (Cadena):** Empezó con comillas y está leyendo todo lo que hay dentro.
- **S4 a S6 (Fecha):** Está leyendo el año, el mes y el día en formato AAAA-MM-DD.
- **S7 (Delimitador):** Encontró un símbolo como {, }, [,], :, , o ;.
- **S8 (Aceptación):** Terminó de leer un token y lo guarda.
- **S9 (Error):** Encontró algo raro y lo anota como error.

Toda esta lógica está escrita en el método siguienteToken() de la clase AnalizadorLexico.



4. La gramática – Reglas del lenguaje TaskScript

Así como el español tiene reglas para ordenar las palabras, TaskScript tiene su propia gramática.

Aquí están las reglas que usamos (las mayúsculas son las palabras exactas que deben aparecer):

`<programa> ::= TABLERO CADENA "{" <lista_columnas> "}" ";"`

`<lista_columnas> ::= <columna> <mas_columnas>`

`<mas_columnas> ::= <columna> <mas_columnas>`

`| ε`

`<columna> ::= COLUMNA CADENA "{" <lista_tareas> "}" ","`

`<lista_tareas> ::= <tarea> <mas_tareas>`

`<mas_tareas> ::= "," <tarea> <mas_tareas>`

`| ε`

`<tarea> ::= tarea ":" CADENA "[" <lista_atributos> "]"`

`<lista_atributos> ::= <atributo> <mas_atributos>`

`<mas_atributos> ::= "," <atributo> <mas_atributos>`

`| ε`

`<atributo> ::= prioridad ":" <prioridad>`

`| responsable ":" CADENA`

`| fecha_limite ":" FECHA`

`<prioridad> ::= ALTA | MEDIA | BAJA`

5. Parser descendente recursivo

El parser es la parte que aplica las reglas de la gramática. Por cada regla (las que vimos arriba) hay una función en el código:

Función	Regla que aplica
analizarPrograma()	<programa>
analizarListaColumnas()	<lista_columnas>
analizarColumna()	<columna>
analizarListaTareas()	<lista_tareas>
analizarTarea()	<tarea>
analizarListaAtributos()	<lista_atributos>
analizarAtributo()	<atributo>
analizarPrioridad()	<prioridad>

Cada función revisa que los tokens aparezcan en el orden que dice la regla. Si algo falla, anota el error y trata de seguir adelante. Mientras va analizando, va construyendo un árbol de derivación, que es como un mapa de cómo se organizaron las reglas para ese archivo.

Por ejemplo, `analizarPrograma()` espera encontrar `TABLERO`, luego una `CADENA`, luego `{`, luego la lista de columnas, luego `}` y finalmente `;`. Si en algún paso no encuentra lo que espera, reporta el error y se sincroniza (salta tokens hasta un lugar seguro).

6. Estrategia de recuperación de errores

Cuando el parser encuentra un error **no se detiene**. Sigue leyendo hasta el final del archivo para poder reportar todos los errores de una sola pasada. Para lograrlo usa dos técnicas:

- **Sincronización:** Si encuentra algo inesperado, avanza rápidamente los tokens hasta encontrar un punto seguro, como un `;`, `}`, `]` o `,`. Así puede retomar el análisis en la siguiente sección.
- **Eliminación de atributos inválidos:** Si dentro de los corchetes de una tarea aparece algo que no es prioridad, responsable o fecha_limite (por ejemplo, color), el parser descarta todo ese atributo (incluido el `:` y el valor) hasta la coma o el cierre del corchete. Así evita que un error pequeño provoque una cascada de errores falsos.

Gracias a esto, el programa puede encontrar varios errores en un mismo archivo y mostrarlos todos juntos en la tabla de errores.

7. Justificación de decisiones de diseño

Durante el desarrollo del proyecto se tomaron varias decisiones que ayudan a que el programa sea más claro y fácil de entender:

- **Corchetes para atributos:** En el ejemplo del PDF los atributos van entre `[ ]`, así que adaptamos el lexer y el parser para que acepten ese formato.
- **Gramática transformada a LL(1):** La gramática original no se podía programar directamente con un parser recursivo. La convertimos eliminando la recursión por la derecha y usando producciones auxiliares como `<mas_columnas>`.
- **Función con map para los tipos de token:** Para convertir un número de tipo de token a su nombre (ejemplo: `TABLERO`, `CORCHETE_IZQ`) usamos un map, que es como un diccionario. Es más corto y fácil de modificar que un switch gigante.

- **Interfaz con Qt:** Qt permite diseñar la ventana arrastrando botones y tablas, y se conecta muy bien con C++. Además genera aplicaciones profesionales con poco esfuerzo.

GRACIAS 😊